Data Science (COL2010) — Homework 4

Module 4: Classification

Required: 96 marks Part I: 36 Part II: 60 Optional bonus: 4 marks

What you may use: a Python/NumPy reference

The functions below are sufficient for the reference solution. You may also use ordinary Python control flow, dictionaries, lists and sets. You do not need to search the entire NumPy API.

Library restriction. Use Python's standard library and NumPy for Tasks 1–24. For Task 25 only, `sklearn.tree.DecisionTreeClassifier` is allowed as a base learner. Do not use a high-level classifier or model-selection routine to replace a requested implementation. The supplied setup, optional Newton safeguard, plotting and comparison helpers may internally use other libraries; their implementation is not assessed. The provided `baseline_predict` helper is allowed only in the Explorations/laboratory, never inside the 25 assessed functions.

Library/function	Use in this homework
`np.asarray, np.array`	Construct arrays; preserve compatible numeric types.
`np.zeros, np.ones, np.full, np.empty_like, np.eye`	Initialize counts, labels, probabilities and regularization matrices.
`np.arange, np.c_, np.column_stack`	Row indices, intercept columns and assembled matrices.
`np.sum, np.mean, np.std, np.var`	Class frequencies, mean losses and population moments (ddof=0).
`np.min, np.max, np.minimum, np.maximum`	Stable score shifts, variance floors, Bayes and hinge losses.
`np.argmax, np.argsort`	Winning class and sorted neighbors; use stable sorting for distance ties.
`np.unique, np.bincount`	Sorted distinct classes/feature values and counts. Beware nonconsecutive labels.
`np.where, np.any, np.clip, np.setdiff1d`	Masks, numerical clipping and out-of-bag rows.
`np.exp, np.log, np.log2, np.logaddexp, np.sqrt`	Likelihoods, entropy, stable logistic loss and kernel calculations.
`np.linalg.norm, np.linalg.solve`	Euclidean distance, gradient norm and Newton linear systems.
`A @ B, A.T`	Matrix multiplication and transposition.
Boolean indexing and broadcasting	Per-class rows and pairwise distances without nested scalar loops.
`np.random.default_rng(seed), rng.integers`	Reproducible bootstrap sampling and forest seeds.
`time.perf_counter`	Optional fit/prediction timing in explorations.

Library/function	Use in this homework
`DecisionTreeClassifier.fit, .predict`	Task 25 base trees only; parameters: max_depth, max_features, random_state.

Files and setup

Use Python $\geq$ 3.10. Install `numpy scikit-learn matplotlib`. Run `python setup.py` from the folder containing all supplied files. Setup generates `module4_data.npz` and thirteen files under `classification_datasets/`. No Internet connection is required: the real data are bundled with scikit-learn. Synthetic data are generated with a fixed seed.

Student files are listed below:

- `setup.py` and `dataset_suite.py`: data generation.
- `inputs.py` and `support.py`: provided helpers.
- `skeleton-solutions.py`: functions to complete.

Copy the skeleton to `solutions.py` and complete all required functions; Tasks 11–12 are optional bonus tasks. The instructor's reference `solutions.py` is not distributed. Submit your completed file and an exploration report with plots and concise observations. The original task allocation totals 100 marks, of which 4 marks (Tasks 11–12) are now optional bonus marks. The required total is therefore 96 marks (Part I: 36; Part II: 60); scale to 100 if desired. Explorations for required tasks are supporting analysis, without additional marks.

Use the revised Task 23 signature and its corresponding revised input helper; otherwise do not change signatures or input helpers. The autograder checks returned values and structures, not printed labels. This revision deliberately removes fragile printed-output requirements. Functions must not mutate their inputs. Inputs are finite, nonempty, and dimensionally compatible unless a task explicitly permits an empty set/count. Labels are nonnegative integers. Parameters follow each task's stated domain; raising errors for invalid inputs is not assessed. A query matrix has at least one row. Tasks 9 and 16 explicitly support scalar/empty-total behavior described below.

How parameters are supplied and how to test

For every Task N there is a matching `get_taskN_inputs()` helper, with arguments in exactly the function signature order. For example:

```
from inputs import get_task4_inputs, get_task23_inputs
posterior = knn_posterior(*get_task4_inputs())
w, history = logistic_gradient_descent(*get_task23_inputs())
```

The supplied examples are convenient local tests. The autograder can use other valid shapes, labels and parameter values. Do not hard-code the supplied K, number of features, class labels or seed. Use small examples of your own as well.

Datasets are obtained by name:

```
from inputs import available_datasets, load_dataset
print(available_datasets())
X, y, V, v, T, t = load_dataset("moons", standardized=True)
# X,y: training; V,v: validation; T,t: held-out test
```

```
k, scores = knn_model_selection(X, y, V, v, (1,3,5,9,15))
# Freeze k before computing test predictions. Do not tune with t.
```

Use raw binary arrays for Bernoulli NB. Standardization parameters are fitted on training data only. The raw and standardized dataset variants retain the same split and labels. For Task 23, append a column of ones to each standardized matrix; other classifiers do not require an intercept column.

Dataset laboratory

Name	Purpose
`gaussian_independent`	Independent Gaussian features; NB assumptions and limited training data.
`gaussian_correlated`	Same original signals plus correlated copies; duplicated evidence.
`gaussian_unequal_variance`	Same class means but different spreads; nonlinear Gaussian NB boundary.
`moons`, `circles`	Nonlinear local/radial geometry; K-NN, NB and kernel comparisons.
`scale_distortion`	Same base signals with unequal scales; raw versus standardized distances.
`irrelevant_dimensions`	Same two signal features plus 80 noise coordinates.
`axis_aligned_rules`	Threshold rules with label noise; depth, pruning and ensembles.
`imbalanced`	Approximately 5% positives; metrics and threshold selection.
`bernoulli_binary`	Binary occurrence features; absent-feature evidence and smoothing.
`breast_cancer`	Real binary medical measurements; class 0 malignant, 1 benign.
`wine`	Real three-class, 13-feature measurements.
`digits`	Real ten-class, 64-feature pixel vectors; stratified subset of 1,000 rows.

Most synthetic cases have 1,000 rows; splits are stratified, approximately 60/20/20. Dataset manifests give exact sizes and sources. Paired base/correlated/scale/noise cases retain identical row splits. For Breast Cancer, relabel both training and evaluation labels if you wish "positive" to mean malignant. Binary precision/recall/F1 must never be applied directly to Wine or Digits; use accuracy and Task 1's multiclass confusion matrix. Macro-averaged metrics are optional, not required new implementations.

Supplied numerical and comparison helpers

```
safeguarded_newton_update(X, y, w, direction, l2)
plot_loss(history, title="Loss")
plot_decision_regions(predict, X, y, title="Decision regions")
baseline_predict(name, X_train, y_train, X_query, **parameters)
```

The Newton safeguard is needed only for the optional Task 23 extension. Plotting is not assessed. The decision-region helper requires a callable mapping a two-dimensional query matrix to labels. Every compared panel must use the same coordinates, bounds and split. Do not project a higher-dimensional model to two features merely to draw its learned boundary.

Baseline names: `linear_svm`, `rbf_svm`, `pruned_tree`, `multiclass_logistic`. SVM parameters include C and gamma; trees include max_depth and ccp_alpha; logistic includes C (inverse regularization strength in this helper). These are instructor-provided solvers for analysis, not functions students must implement. Example:

```
pred = baseline_predict("rbf_svm", X, y, V, C=1.0, gamma=0.5)
# Compare several settings on V,v, then evaluate the frozen one on T,t.
```

Learning outcomes and evaluation conventions

Implement class evaluation, ordinary K-NN, Gaussian/Bernoulli NB, logistic gradient descent, softmax loss, SVM objective/kernel computations, greedy trees and bootstrap/OOB forests. Logistic Hessian and Newton/IRLS work are optional. Explain their assumptions and compare generalization, geometry and computational cost.

All required logistic losses use a *mean*; required Task 23 is unregularized. Optional Tasks 11–12 use mean loss plus $\lambda\|w_{1:}\|^2/2$. SVM Task 14 uses *summed* hinge loss with C. Tree entropy uses base 2; all likelihood and classification losses use natural logarithms. Sorted original labels resolve class ties. Validation data choose hyperparameters/thresholds; test data are reserved for final evaluation of frozen choices. If OOB results guide tuning, they are also selection data. Finite-sample rankings need not match a universal ordering.

Output types and notation

Every task has an explicit Expected output: block. Return the specified Python object; printing it is not a substitute for returning it. A tuple has the stated element order. Dictionary keys are strings; values have the individual types listed. All numerical arrays are NumPy arrays: integer counts/labels/indices use an integer dtype; probabilities, parameters, distances and losses use a floating dtype. The grader does not require a particular integer bit width. A Python float means a scalar float, not a length-one array.

Unless a task defines otherwise, N is the number of training rows, M the number of other/query rows, d the number of feature columns, and C the number of classes. For logistic tasks, d includes the intercept column. Notation (d,) means a one-dimensional array, not a column matrix (d,1). Empty arrays are permitted only where the task explicitly specifies them.

Part I: Foundational components (40 marks)

Task 1: Confusion matrix [2 marks]

Description. Return an integer $C \times C$ matrix. Entry (i, j) counts examples whose actual class is `classes[i]` and predicted class is `classes[j]`. Rows are actual, columns predicted. `classes` is sorted, distinct, and includes every supplied label.

```
def confusion_matrix(y_true, y_pred, classes):
    pass
```

Expected output:

```
np.ndarray
```

Output description: An integer array of shape (C, C), where C is the number of supplied classes. Entry $[i, j]$ is the count of actual class classes[i] predicted as classes[j]. Each example contributes to exactly one entry.
Example arguments: `get_task1_inputs()`.

Task 2: Binary evaluation metrics [2 marks]

Description. Labels are 0/1, with 1 positive. Return a dictionary with string keys `accuracy`, `precision`, `recall`, `f1`. Use $TP/(TP + FP)$, $TP/(TP + FN)$, and $2TP/(2TP + FP + FN)$. A zero denominator gives 0. Accuracy uses all examples.

```
def binary_metrics(y_true,y_pred):
    pass
```

Expected output:

```
dict[str, float]
```

Output description: A dictionary with exactly four string keys: accuracy, precision, recall and f1. Each value is a Python float between 0 and 1. These respectively give the fraction correct, positive predictive value, positive-class sensitivity and harmonic balance of precision/recall. Undefined precision/recall/F1 due to a zero denominator is returned as 0.0.
Example arguments: `get_task2_inputs()`.
Exploration. For 99 negatives and one positive, predict all negatives. Explain the difference between accuracy and F1; neither score alone describes every application.

Task 3: Training-only standardization [2 marks]

Description. Return `(X_train_std, X_other_std, mean, scale)`. Compute population standard deviations (`ddof=0`) using training rows only. Replace a zero standard deviation with 1. Apply those same parameters to the other rows. Do not modify either input.

```
def standardize_train_apply(X_train,X_other):
    pass
```

Expected output:

```
tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]
```

Output description: A four-element tuple, in this exact order: (1) standardized training data, a float array of shape (N, d); (2) standardized other data, a float array of shape (M, d); (3) training feature means, a float array of shape $(d,)$; (4) training feature scales, a float array of shape $(d,)$. A constant training feature has scale 1.0. N and M may differ.
Example arguments: `get_task3_inputs()`.
Exploration. Compare raw and standardized K-NN on `scale_distortion`. Verify that validation/test feature means need not be zero. Explain why fitting their scaling separately changes the distance geometry.

Task 4: Ordinary K-NN posterior [2 marks]

Description. For a single query, choose the K nearest training rows using Euclidean distance. Return one class fraction per entry of sorted `classes`. Distance ties use lower training-row index first. Valid $1 \leq K \leq N$. The fractions must sum to one.

```
def knn_posterior(x,X_train,y_train,K,classes):
    pass
```

Expected output:

```
np.ndarray
```

Output description: A float array of shape $(C,)$ containing the fractions of the selected K neighbors belonging to each supplied class, in classes order. The entries lie in $[0, 1]$ and sum to 1. Return these probabilities.
Example arguments: `get_task4_inputs()`.
Exploration. Compare posteriors with the same winning class but different confidence. Explain why small K can be sensitive to one noisy label.

Task 5: Bayes error and limiting 1-NN error [2 marks]

Description. Input `p` is an array of hypothetical true $P(Y = 1 \mid x)$ values in $[0, 1]$. Return two arrays: `(bayes_error, one_nn_error)` with entries $\min(p, 1-p)$ and $2p(1-p)$. The second expression assumes an asymptotically local neighbor and conditionally independent query/neighbor labels; it is not a finite-sample empirical error estimate.

```
def bayes_and_one_nn_error(p):
    pass
```

Expected output:

```
tuple[np.ndarray, np.ndarray]
```

Output description: A two-element tuple (bayes_error, one_nn_error). Both are float arrays with exactly the same shape as the input p. The first gives minimum achievable conditional classification error at each p; the second gives the hypothetical limiting conditional 1-NN error under the stated assumptions. These are pointwise errors, not averages over p.
Example arguments: `get_task5_inputs()`.
Exploration. Check the bound $R_{1NN}(x) \leq 2R^*(x)$ at the supplied probabilities. Why can 1-NN still err when the neighbor is arbitrarily close?

Task 6: Gaussian log-likelihood [2 marks]

Description. For one observation with independent Gaussian coordinates, return the summed log density $\sum_j [-\frac{1}{2}\log(2\pi v_j) - \frac{1}{2}(x_j - \mu_j)^2/v_j]$. Floor each variance at 10^{-9}. A density is not a class posterior probability.

```
def gaussian_log_likelihood(x,mean,var):
    pass
```

Expected output:

```
float
```

Output description: A single Python float giving the sum of the per-feature Gaussian log densities for the observation. It is a natural-log density score, not a probability; it need not lie in $[0, 1]$ and may be positive or negative.
Example arguments: `get_task6_inputs()`.
Exploration. Explain why log-space computation preserves the winning class and avoids underflow. Which assumption permits summing separate coordinate log densities?

Task 7: Gaussian NB parameter estimation [2 marks]

Description. Return a dictionary: `classes` (sorted labels), `prior` (C), `mean` ($C \times d$), `var` ($C \times d$). Use empirical priors and within-class population variances, floored at 10^{-9}. Every training class has at least one row. Integer labels need not be consecutive.

```
def gaussian_nb_parameters(X,y):
    pass
```

Expected output:

```
dict[str, np.ndarray]
```

Output description: A dictionary with exactly four keys. classes: integer array of shape $(C,)$ with sorted original labels. prior: float array of shape $(C,)$ with empirical class probabilities, summing to 1. mean: float array of shape (C, d), one class mean per row. var: float array of shape (C, d), one class population variance per feature, floored at 10^{-9}. Row k in every parameter array corresponds to classes[k].
Example arguments: `get_task7_inputs()`.
Exploration. Compare the number of variance parameters with a full covariance model. What information does Gaussian NB discard in `gaussian_correlated`?

Task 8: Smoothed Bernoulli NB parameters [2 marks]

Description. Features are binary 0/1 and $\alpha > 0$. Return `classes`, `prior`, and `prob` ($C \times d$). Each feature probability is $(N_{cj}^{(1)} + \alpha)/(N_c + 2\alpha)$. Priors are empirical and unsmoothed.

```
def bernoulli_nb_parameters(X,y,alpha=1.):
    pass
```

Expected output:

```
dict[str, np.ndarray]
```

Output description: A dictionary with exactly three keys. classes: sorted original labels, an integer array of shape $(C,)$. prior: empirical class probabilities, a float array of shape $(C,)$. prob: a float array of shape (C, d); entry $[k, j]$ is the smoothed probability that feature j is 1 in class classes[k]. These feature probabilities lie strictly between 0 and 1; their sum across features need not be 1.
Example arguments: `get_task8_inputs()`.
Exploration. Make a feature absent in all training rows of one class. Compare unsmoothed probability with $\alpha \in \{0.1, 1, 5\}$. Explain the role of the denominator $N_c + 2\alpha$.

Task 9: Numerically stable sigmoid [2 marks]

Description. Return an array with the same shape as `z`, containing $\sigma(z)$. Use the positive/negative branches $1/(1+e^{-z})$ and $e^z/(1+e^z)$ respectively, so no large positive exponential is evaluated. A scalar input may return a zero-dimensional array. Do not clip the scores.

```
def stable_sigmoid(z):
    pass
```

Expected output:

```
np.ndarray
```

Output description: A float array with the same shape as z, containing the sigmoid of each input entry. A scalar z returns a zero-dimensional NumPy array (shape ()). Values are in $[0, 1]$ in floating-point arithmetic; sufficiently extreme inputs may round to exactly 0 or 1. Do not return thresholded class labels.
Example arguments: `get_task9_inputs()`.
Exploration. Compute sigmoid and its derivative at $-10, 0, 10$. Relate these values to saturation and the probability boundary at 0.5.

Task 10: Mean binary cross-entropy [2 marks]

Description. Return $-N^{-1}\sum_n[y_n \log p_n + (1-y_n)\log(1-p_n)]$. Clip probabilities into $[10^{-12}, 1-10^{-12}]$ before the logarithm. Targets are 0/1. We consistently use the mean loss in logistic tasks..

```
def binary_cross_entropy(y,p):
    pass
```

Expected output:

```
float
```

Output description: A single Python float: the average clipped-probability binary cross-entropy across all examples. It is nonnegative and uses natural logarithms. It is not a per-example loss array or the sum of losses.
Example arguments: `get_task10_inputs()`.
Exploration. Compare a confident correct prediction with a confident wrong prediction. Explain how converting a sum to a mean changes scale but not the unregularized minimizer.

Task 11: Regularized logistic gradient [Optional — 2 bonus marks]

Description. `X` includes a first column of ones; `w` includes its intercept. Compute probabilities internally. Return $X^T(p-y)/N + \lambda D w$, where $D = \mathrm{diag}(0, 1, \ldots, 1)$ and `l2` is $\lambda \geq 0$. This differentiates mean cross-entropy plus $\lambda \sum_{j>0} w_j^2/2$.

```
def logistic_gradient(X,y,w,l2=0.):
    pass
```

Expected output:

```
np.ndarray
```

Output description: A float array of shape $(d,)$ containing the gradient with respect to every weight in w, in the same order. The first component is the derivative with respect to the intercept. Remaining components include the L2 penalty contribution. Here d includes the intercept column.
Example arguments: `get_task11_inputs()`.
Exploration. For `gaussian_correlated`, discuss how L2 can stabilize coefficients. Explain why the intercept penalty is excluded.

Task 12: Regularized logistic Hessian [Optional — 2 bonus marks]

Description. Return $H = X^T \operatorname{diag}(p_n(1-p_n))X/N + \lambda D$, using the same intercept and mean-loss conventions as Task 11. Do not build a large $N \times N$ diagonal matrix if broadcasting can avoid it.

```
def logistic_hessian(X,w,l2=0.):
    pass
```

Expected output:

```
np.ndarray
```

Output description: A float array of shape (d, d) containing the symmetric regularized Hessian, with rows and columns ordered as w. The first row/column corresponds to the intercept. It represents curvature of the mean-loss objective, not its inverse or a Newton direction.
Example arguments: `get_task12_inputs()`.
Exploration. Use the quadratic form $v^T H v$ to explain positive semidefiniteness. Why does convexity alone not guarantee a finite unregularized optimum on separable data?

Task 13: Softmax probabilities and loss [2 marks]

Description. For X $(N \times d)$, W $(d \times C)$, and targets `t` in $0, \ldots, C-1$, return `(mean_loss, probabilities)`. Stabilize scores by subtracting the row maximum. Compute loss with log-sum-exp, not by clipping tiny probabilities. Probability rows sum to one. No softmax optimizer is required.

```
def softmax_loss_probabilities(X,W,t):
    pass
```

Expected output:

```
tuple[float, np.ndarray]
```

Output description: A two-element tuple (mean_loss, probabilities). mean_loss is a Python float giving average multiclass cross-entropy. probabilities is a float array of shape (N, C); row n contains the class probabilities for input n, ordered by weight-matrix columns. Each row sums to 1. Target t[n] indexes the relevant column.
Example arguments: `get_task13_inputs()`.
Exploration. Add a different constant to every class score within each row. Verify probabilities are unchanged. Use the provided multiclass logistic baseline on Wine or Digits for the comparison laboratory.

Task 14: Soft-margin SVM objective [2 marks]

Description. Targets are $-1\& +1$, the intercept is separate, and $C > 0$. Return `(hinge_sum, objective)` with $h_n = \max(0, 1 - y_n(X_n w + b))$ and objective $\|w\|^2/2 + C\sum_n h_n$. Use a sum, not a mean. No SVM training solver is required.

```
def svm_primal_objective(X,y_pm1,w,b,C):
    pass
```

Expected output:

```
tuple[float, float]
```

Output description: A two-element tuple (hinge_sum, objective), both Python floats. The first is the sum of all nonnegative hinge penalties. The second adds the squared-weight regularizer to C times that sum.
Example arguments: `get_task14_inputs()`.
Exploration. Identify points with margins above 1, between 0 and 1, and below 0. Discuss geometric margin versus functional margin and the effect of C. For hypothetical dual coefficients $\alpha = (0, 0.3, 0, 0.8, 0)$, identify the support-vector indices and explain why only nonzero coefficients contribute to the final boundary.

Task 15: RBF kernel matrix [2 marks]

Description. Return an $N \times M$ matrix $K_{ij} = \exp(-\gamma\|X_i - Z_j\|^2)$ for $\gamma > 0$. Clamp small negative squared distances caused by roundoff to 0. Z may differ from X.

```
def rbf_kernel(X,Z,gamma):
    pass
```

Expected output:

```
np.ndarray
```

Output description: A float array of shape (N, M), where N is the number of rows in X and M the number in Z. Entry $[i, j]$ is the RBF similarity between X[i] and Z[j]. Values lie in $[0, 1]$ in floating-point arithmetic. When X=Z, it is a square symmetric Gram matrix with diagonal entries 1.
Example arguments: `get_task15_inputs()`.
Exploration. For $Z = X$, check symmetry and diagonal entries. Use the provided RBF SVM baseline on Moons/Circles with validation-selected C and gamma. Explain the quadratic storage cost when $N = M$.

Task 16: Node impurity [2 marks]

Description. Given nonnegative class counts, return `(gini, entropy)` with $1 - \sum p_k^2$ and $-\sum_{p_k>0} p_k \log_2 p_k$. If total count is 0, return `(0.0,0.0)`. Treat $0 \log 0$ as 0.

```
def node_impurities(class_counts):
    pass
```

Expected output:

```
tuple[float, float]
```

Output description: A two-element tuple (gini, entropy), both Python floats, giving node Gini impurity and base-2 Shannon entropy. Return (0.0,0.0) for a zero-total count vector. These are two different impurity measures, not an impurity and a gain.
Example arguments: `get_task16_inputs()`.
Exploration. Compare pure, balanced and skewed nodes. Explain using concavity why a weighted child impurity cannot exceed the parent impurity.

Task 17: Best greedy Gini split [2 marks]

Description. Suppose you are building a decision tree and have reached a node containing $N > 0$ rows, given as a feature matrix `X` of shape (N, d) and labels y of shape $(N,)$. To grow the tree, you must choose a single question of the form "is x_j $\leq \theta$?" that best separates the classes. Rows that satisfy the condition go to the left child, and the rest go to the right child. Return `(feature, threshold, gain)`. Try each feature in increasing index order and midpoints between its sorted distinct values in increasing order. Split with $x_j \leq \theta$ left. Gain is parent Gini minus size-weighted child Gini. Accept only gains above 10^{-12}; improvements must exceed the current best by 10^{-12}, so ties keep the first candidate. If no qualifying split exists, return `(None,None,0.0)`. No child may be empty.

```
def best_gini_split(X,y):
    pass
```

Expected output:

```
tuple[int | None, float | None, float]
```

Output description: A three-element tuple (feature, threshold, gain). For a valid improving split, feature is a Python integer column index, threshold a Python float, and gain a positive Python float measuring reduction in Gini impurity. If no gain exceeds the specified tolerance, return exactly (None,None,0.0), including for pure nodes or constant features.
Example arguments: `get_task17_inputs()`.
Exploration. Compute the supplied one-dimensional split by hand. Why can a locally optimal split fail to produce a globally optimal tree?

Task 18: Bootstrap and out-of-bag indices [2 marks]

Description. Suppose you want to train a Random Forest and you have `n_samples`> 0 samples (Henceforth, consider `n_samples` $= N$). To implement bootstrapping, you must sample an index $\in \{0, 1, ..., N\text{-}1\}$ randomly N times **with replacement**. The indices that were not sampled form the Out of Bag Indices and serve as a convenient validation set. For uniformity in submissions, initialize the random generator using seed, then call `rng.integers(0,N,N)`. Return `(bootstrap, oob)`. Bootstrap has length N and permits repeated indices. OOB is the sorted array of original row indices absent from the draw.

```
def bootstrap_oob(n_samples,seed=42):
    pass
```

Expected output:

```
tuple[np.ndarray, np.ndarray]
```

Output description: A two-element tuple (bootstrap, oob) of integer arrays. bootstrap has shape $(N,)$ and contains N sampled row indices, possibly repeated and in draw order. oob has shape $(U,)$ and contains the sorted distinct original indices never sampled. U may be 0, in which case oob is an empty integer array of shape $(0,)$. OOB indices are not labels or a Boolean mask.
Example arguments: `get_task18_inputs()`.
Exploration. Repeat with several seeds and N. Compare average OOB fraction with $(1 - 1/N)^N$ and its limit e^{-1}.

Task 19: Multiclass majority vote [2 marks]

Description. The following experiment was conducted: N classification models were taken and M different inputs (queries) were given to them. Each prediction can be seen in the `predictions` array, which has the shape $N \times M$, one model per row and one query per column. To classify a query, you decide to take the majority class predicted for each query. Return M class labels. In case of equal number of votes, the smallest class wins the tie.

```
def majority_vote(predictions,classes):
    pass
```

Expected output:

```
np.ndarray
```

Output description: An integer array of shape $(M,)$, containing one majority-vote original class label per query column.
Example arguments: `get_task19_inputs()`.
Exploration. Construct a tie with nonconsecutive labels such as 2 and 5. Explain why returning the argmax column index instead of its class label is incorrect.

Task 20: Validation threshold selection [2 marks]

Description. For binary validation labels and probabilities, evaluate F1 using `p_validation >= threshold`. Return `(best_threshold, f1_scores)` in the supplied threshold order. On tied scores select the first threshold. Thresholds are a nonempty ordered sequence in $[0, 1]$.

```
def select_f1_threshold(y_validation,p_validation,thresholds):
    pass
```

Expected output:

```
tuple[float, np.ndarray]
```

Output description: A two-element tuple (best_threshold, f1_scores). best_threshold is a Python float equal to the first supplied threshold attaining the highest validation F1. f1_scores is a float array of shape $(S,)$, in the same order as the S supplied thresholds.
Example arguments: `get_task20_inputs()`.
Exploration. On `imbalanced`, compare the validation-selected threshold with 0.5, then freeze it before test evaluation. Discuss the precision/recall trade-off; precision need not vary monotonically on a finite sample.

Part II: Complete classifiers (60 marks)

Task 21: K-NN model selection [12 marks]

Description. Implement ordinary K-NN prediction and select K using validation accuracy. Train your KNN classifier using the `X_train, y_train` datasets using each K from `k_values` and then compute the accuracy for `X_validation, y_validation` datasets. Return `(best_k, validation_accuracies)` in the supplied K order, selecting the first best score. Reuse Task 4 and original class labels. In order to break ties between classes with equal K-NN score, choose the class with the smallest value.

```
def knn_model_selection(X_train,y_train,X_validation,y_validation,k_values):
    pass
```

Expected output:

```
tuple[int, np.ndarray]
```

Output description: A two-element tuple (best_k, validation_accuracies). best_k is a Python integer equal to the first supplied K attaining the highest validation accuracy. validation_accuracies is a float array of shape $(L,)$, in the same order as the L supplied k_values.
Example arguments: `get_task21_inputs()`.
Exploration. Compare independent Gaussian, Moons and irrelevant-dimension data using K in {1,3,5,9,15,31}. Freeze K before evaluating test accuracy and F1. Explain the bias/variance and prediction-cost differences. Use only binary datasets for the required F1 comparison.

Task 22: Complete Naive Bayes prediction [12 marks]

Description. Return `(predictions, posteriors, classes)` for each query. `kind` is either `gaussian` or `bernoulli`. Fit priors and parameters on training rows only; add log prior to feature log likelihoods; normalize shifted log scores to posteriors. Bernoulli scores must include both present and absent features. Predictions are original sorted class labels, with smallest-label ties. Bernoulli inputs must remain binary; Gaussian variances use Task 7 flooring.

```
def naive_bayes_predict(X_train,y_train,X_query,kind='gaussian',alpha=1.):
    pass
```

Expected output:

```
tuple[np.ndarray, np.ndarray, np.ndarray]
```

Output description: A three-element tuple (predictions, posteriors, classes).
predictions: integer array of shape $(M,)$ with original winning class labels for M queries.
posteriors: float array of shape (M, C) with normalized class probabilities, each row summing to 1.
classes: sorted training class labels, an integer array of shape $(C,)$.
Column k in posteriors corresponds to classes[k]; predictions must use those labels, not column indices.
Example arguments: `get_task22_inputs()`.
Exploration. Compare Gaussian NB on independent and correlated data using identical splits and training subsets of 40, 100 and all available rows (include every class). Also test unequal-variance data and Bernoulli NB on binary data with several alpha values. Explain generative versus discriminative modeling and confidence as well as accuracy.

Task 23: Logistic regression using gradient descent [12 marks]

Required description. Train binary logistic regression by minimizing mean binary cross-entropy using *batch gradient descent*. Labels are 0/1 and `X` includes an intercept column. Initialize weights to zero. For each iteration, compute $z = Xw$, $p = \sigma(z)$, and the gradient of the *unregularized* mean loss

$$g = \frac{1}{N} X^T (p - y).$$

Update $w \leftarrow w - \eta g$ using the supplied positive learning rate $\eta =$ `learning_rate`. Stop before updating if $\|g\|_2 \leq$ `tol`, or after at most `max_steps` updates. Record the initial loss and the loss after every update. The loss at weights w is

$$L(w) = \frac{1}{N} \sum_{n=1}^{N} \left[\log(1 + e^{z_n}) - y_n z_n \right], \qquad z = Xw.$$

Use `np.logaddexp(0,z)` for numerical stability. No Hessian, Newton system, IRLS, or regularization is required. Use a suitably small learning rate; convergence for every possible input/learning-rate combination is not guaranteed.

```
def logistic_gradient_descent(X,y,learning_rate=0.1,max_steps=500,tol=1e-6):
    pass
```

Expected output:

```
tuple[np.ndarray, np.ndarray]
```

Output description: Return `(w, loss_history)`. `w` is a float array of shape $(d,)$, including the intercept at index 0. `loss_history` is a one-dimensional float array of length $u + 1$, where $0 \leq u \leq$ `max_steps` is the number of updates. Its first entry is the loss at zero weights; its final entry is the loss at the returned weights.

Example arguments: `get_task23_inputs()` (update this helper to supply the new signature).

Required exploration. Compare several learning rates on `gaussian_independent` and `gaussian_correlated`; plot loss versus iteration and compare training/validation error. On `moons`, explain the limitation of a linear decision boundary.

Optional extension (not required for Task 23). Implement the regularized Newton method from the previous version of Task 23: use the optional gradient and Hessian from Tasks 11–12, solve $H\Delta = g$, apply `safeguarded_newton_update`, and compare convergence against gradient descent. Explain the IRLS interpretation. This extension is not part of the required `logistic_gradient_descent` function or its grading.

Task 24: Greedy classification tree [12 marks]

Description. Return `(tree, predictions)`. Implement recursive growth using Task 17. The root has depth 0. Stop at `max_depth`, below `min_samples_split`, at a pure node, or if no positive-gain split exists. Every node stores `prediction`, the local majority original label (smallest-label ties). An internal node additionally stores `feature`, `threshold`, `left`, `right`. Leaves have only `prediction`. Route each query recursively or iteratively to its leaf. Nonnegative depth and minimum split size at least 2 are valid.

```
def decision_tree_train_predict(X_train,y_train,X_query,max_depth=4,min_samples_split=2):
    pass
```

Expected output:

```
tuple[dict[str, object], np.ndarray]
```

Output description: A two-element tuple (tree, predictions).
tree is a nested Python dictionary.
Every node has prediction: a Python integer giving its local majority original class. A leaf has only this key. An internal node additionally has feature (Python int), threshold (Python float), left (child dictionary) and right (child dictionary).
predictions is an integer array of shape $(M,)$ containing one predicted original class label per query row.
Example arguments: `get_task24_inputs()`.
Exploration. On axis-aligned rules and Moons, sweep depths {1,2,4,8}; plot training/validation error. Use the supplied pruning baseline with validation-selected `ccp_alpha`; implementing weakest-link pruning is not required. Explain instability after resampling and why greedy growth can miss interactions with zero initial impurity gain.

Task 25: Random forest and OOB evaluation [12 marks]

Description. Use `DecisionTreeClassifier` only as the base tree; implement sampling, voting and OOB bookkeeping yourself. Return a dictionary with `predictions` (M labels), `oob_predictions` (N labels, -1 for uncovered rows), `oob_counts` (N counts), `oob_accuracy` (covered rows only, NaN if none), `oob_coverage` (covered fraction). Classes are nonnegative integers, so -1 is reserved. The number of trees is a positive integer; max_depth is None or a positive integer. Use a master default_rng(seed); for each tree draw bootstrap seed, then tree seed, each from integers(0,$2^{31}-1$). Call Task 18 with the bootstrap seed. Set the base tree random_state to the tree seed. If `max_features=None`, use $\max(1, \lfloor\sqrt{d}\rfloor)$; otherwise use the supplied integer from 1 to d. Feature randomization occurs at every split through the base tree. Skip empty OOB prediction calls. Aggregate query votes from all trees and each training row only from its OOB trees, with smallest-label ties. Never use y of a queried row to vote.

```
def forest_oob(X,y,X_query,n_estimators=30,max_depth=None,max_features=None,seed=42):
    pass
```

Expected output:

```
dict[str, object]
```

Output description: A dictionary with exactly five keys.
predictions: integer array of shape $(M,)$, one all-tree majority label per query.
oob_predictions: integer array of shape $(N,)$, one OOB-only majority label per training row, or -1 when uncovered.
oob_counts: integer array of shape $(N,)$, counting how many trees omitted each training row.
oob_accuracy: Python float, fraction correct among covered training rows only; return float NaN if none is covered.
oob_coverage: Python float in $[0, 1]$, the fraction of training rows with at least one OOB vote. The labels in both prediction arrays are original class labels, except the reserved -1 sentinel.
Example arguments: `get_task25_inputs()`.
Exploration. Compare bagging (max_features=d) and forests (max_features=$\lfloor\sqrt{d}\rfloor$) on correlated and axis-aligned datasets. Sweep tree counts {1,5,15,30,50}. Report OOB accuracy and coverage, validation accuracy and prediction time. Explain the correlated-learner variance floor; an OOB score used for tuning is not an untouched final test score.

Final comparison report

Organize the Explorations into a compact report. Include: (i) raw/standardized and low/high-dimensional K-NN comparisons; (ii) independent/correlated/unequal-variance NB and logistic comparisons; (iii) linear versus nonlinear models on Moons/Circles; (iv) tree depth/pruning and bagging/forest results; (v) imbalance threshold/metric results; (vi) at least one real binary and one real multiclass experiment.

For required multiclass experiments, use your K-NN, NB, tree or forest plus the provided multiclass logistic baseline. Do not implement an untaught multiclass optimizer. Report validation-selected settings, final test accuracy and confusion matrices; for binary experiments also report precision, recall and F1. Measure fit and prediction time separately with the same query count. K-NN stores training examples rather than fitting parameters. Give a short explanation grounded in each model's assumptions. Use several reproducible resamples when arguing about variance; a single run is not proof. Test results should be computed after choices are frozen, not used to redesign them.